



Algorithm III – Week 1 Lesson Plan



Topic: Pointer Consolidation and Dynamic Memory



Lesson Objectives

By the end of this 3-hour theory session, students should be able to:

Knowledge

- Explain the difference between a value, an address, a pointer, and a dereferenced value.
- Explain how pointer parameters allow a function to access or modify caller data.
- Explain the basic relationship between arrays and pointers without ignoring array bounds.
- Describe why dynamic memory is needed when the required size is known only at runtime.
- Explain the purpose of `malloc`, `calloc`, `free`, and `NULL` in C.
- Explain ownership, allocation lifetime, memory leaks, dangling pointers, invalid access, and allocation failure.

Skills

- Trace pointer values and the variables they reference step by step.
- Pass addresses to functions and predict changes made through pointer parameters.
- Allocate one-dimensional arrays with `malloc` and `calloc` using a safe size expression.
- Check allocation results before using dynamically allocated memory.
- Release dynamic memory exactly once and reset the owning pointer to `NULL`.
- Identify and correct unsafe memory code before running it.

Attitude

- Treat memory safety as part of algorithm correctness, not as an optional extra.
 - Trace and explain memory behavior before trying to shorten the code.
 - Take responsibility for allocation, use, and release of every dynamic block.
 - Prefer clear ownership and lifetime rules over clever pointer manipulation.
-

Lesson Structure (3 Hours)

◇ Part 1: Diagnostic Review and Memory Model (15 mins)

Teacher Action

Begin with a quick diagnostic rather than reteaching every pointer detail. Ask students to explain the following code using value-and-address language:

```
int x = 10;  
int *p = &x;  
*p = 25;
```

Ask students:

- What is stored in **x**?
- What is stored in **p**?
- What does ***p** mean?
- Why does assigning **25** through ***p** change **x**?

Key Bridge

- Algorithm II introduced pointer syntax and basic array traversal.
- Algorithm III now treats pointers as a tool for controlling memory lifetime safely.

“A pointer tells us where data lives. Dynamic memory adds a second responsibility: deciding how long that data should live.”

◇ Part 2: Consolidating Address, Dereference, and Pointer Parameters (20 mins)

Core Meaning

Expression	Meaning	Example result
x	Value stored in the variable	10
&x	Address of the variable	A memory address
p	Address stored in the pointer	Same address as &x
*p	Value found at that address	10

Pointer Parameter Example

```
#include <stdio.h>

void addFive(int *value)
{
    *value = *value + 5;
}

int main(void)
{
    int score = 12;
    addFive(&score);
    printf("%d\n", score);
    return 0;
}
```

Trace the call in this order:

1. `&score` sends the address of `score`.
2. The parameter `value` stores that address.
3. `*value` refers to the caller's original variable.
4. The function changes `score` from 12 to 17.

Safety Reminder

A pointer parameter is safe only when the caller passes a valid address and the function respects the object's valid lifetime and bounds.

◇ Part 3: Arrays, Pointers, and Valid Bounds (20 mins)

```
int values[4] = {3, 6, 9, 12};
int *p = values;

printf("%d\n", *p);           /* 3 */
printf("%d\n", *(p + 2));    /* 9 */
```

Relationship

- `values` can be used as the address of the first element in many expressions.
- `p = values` makes `p` point to `values[0]`.
- `values[i]` and `*(p + i)` access the same element while `i` is valid.
- For an array of length 4, valid indexes are 0 through 3.

Important Boundary Rule

The address one position past the array may be formed for comparison, but it must not be dereferenced. Therefore, `p + 4` may mark the stopping position, while `*(p + 4)` is invalid access.

Mini Trace

<code>i</code>	<code>p + i</code> points to	<code>*(p + i)</code>
0	<code>values[0]</code>	3
1	<code>values[1]</code>	6
2	<code>values[2]</code>	9
3	<code>values[3]</code>	12

◇ Part 4: Why Dynamic Memory Is Needed (25 mins)

Problem Situation

A fixed array requires its size to be decided in advance. Many programs learn the required size only while running—for example, after the user enters the number of students, measurements, or transactions.

```
int n;
printf("How many scores? ");
scanf("%d", &n);

/* The program now needs storage for n integers. */
```

Automatic and Dynamic Lifetime

Storage idea	Created	Lifetime	Released
Local/automatic variable	When execution enters its block	Until the block ends	Automatically
Dynamic allocation	When <code>malloc</code> or <code>calloc</code> succeeds	Until the program calls <code>free</code>	Explicitly by the owner

In common implementations, local variables are associated with the stack and dynamic allocations with the heap. The key algorithmic distinction is not the name of the region; it is who controls the lifetime.

“Dynamic memory is useful because its size and lifetime are controlled at runtime—but that control creates responsibility.”

◇ Part 5: Allocating with malloc and calloc (25 mins)

malloc: Allocate Uninitialized Storage

```
int *scores = malloc(n * sizeof *scores);
```

Read the expression from right to left:

- `sizeof *scores` gives the number of bytes required for one element.
- `n * sizeof *scores` gives the total requested number of bytes.
- `malloc(...)` requests one contiguous block.
- The returned starting address is stored in `scores`.

The contents returned by `malloc` are indeterminate. The program must assign values before reading them.

calloc: Allocate and Zero the Bytes

```
int *counts = calloc(n, sizeof *counts);
```

- `calloc` receives the number of elements and the size of each element separately.
- It initializes the allocated bytes to zero.
- For a beginner integer-array example, the elements begin as zero.

Avoid This Fragile Form

```
int *scores = malloc(n * sizeof(int));
```

This is valid, but `sizeof *scores` is often easier to maintain because it remains correct if the pointer's element type changes.

◇ Part 6: Allocation Failure and Safe First Use (20 mins)

```
int *scores = malloc(n * sizeof *scores);

if (scores == NULL)
{
    printf("Memory allocation failed.\n");
    return 1;
}
```

Key Rule

Never index or dereference a newly returned pointer until the program has confirmed that it is not NULL.

Validation Order

1. Validate the requested element count, such as ensuring `n > 0`.

2. Request the allocation.
3. Check whether the returned pointer is **NULL**.
4. Only then read or write elements.

Example with Input Validation

```
if (n <= 0)
{
    printf("The size must be positive.\n");
    return 1;
}

int *scores = malloc(n * sizeof *scores);
if (scores == NULL)
{
    printf("Not enough memory.\n");
    return 1;
}
```

◇ Part 7: Ownership, Lifetime, free, and NULL (20 mins)

Ownership Question

For every successful allocation, the program must be able to answer: “Which function or part of the program is responsible for releasing this block?”

```
int *scores = malloc(n * sizeof *scores);
if (scores == NULL)
{
    return 1;
}

/* Use scores only while the allocation is alive. */

free(scores);
scores = NULL;
```

Meaning of Each Step

- **free(scores)** ends the lifetime of the dynamically allocated block.
- After **free**, the old address must not be dereferenced or indexed.
- The pointer variable still contains the old address until it is changed.
- Assigning **NULL** makes accidental later checks safer and prevents reuse through that pointer.
- **free(NULL)** is safe and has no effect.

Aliasing Warning

If another pointer also stored the same address, setting only the owner to `NULL` does not update that alias. The alias becomes dangling and must not be used.

♦ Part 8: Recognising Unsafe Memory States (20 mins)

Problem	Example	Why unsafe
Uninitialized pointer	<code>int *p; *p = 5;</code>	<code>p</code> does not identify a valid object.
NULL dereference	<code>int *p = NULL; printf("%d", *p);</code>	<code>NULL</code> points to no object.
Out-of-bounds access	<code>scores[n]</code>	The last valid index is <code>n - 1</code> .
Memory leak	Allocate, then lose the only pointer	The block remains allocated but unreachable.
Dangling pointer	Use <code>scores</code> after <code>free(scores)</code>	The allocation's lifetime has ended.
Double free	Call <code>free(scores)</code> twice	The second release is undefined behavior.

Debug Before Running

```
int *data = malloc(5 * sizeof *data);
for (int i = 0; i <= 5; i++)
{
    data[i] = i * 2;
}
free(data);
printf("%d\n", data[0]);
free(data);
```

Students should identify three separate defects: `i <= 5` goes out of bounds, `data[0]` is accessed after `free`, and the same block is freed twice.

◇ Part 9: Integrated Trace, Guided Practice, and Exit Check (15 mins)

Integrated Example

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n;
    printf("Number of values: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid size.\n");
        return 1;
    }

    int *values = malloc(n * sizeof *values);
    if (values == NULL)
    {
        printf("Allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &values[i]);
    }

    int sum = 0;
    for (int i = 0; i < n; i++)
    {
        sum += values[i];
    }

    printf("Sum = %d\n", sum);

    free(values);
    values = NULL;
    return 0;
}
```

Student Explanation Sequence

1. What information is known before allocation?
2. What exact number of bytes is requested?

3. Which pointer owns the allocation?
4. When does the allocation become safe to use?
5. What indexes are valid?
6. Where does the allocation's lifetime end?

Evaluation Questions

7. What is the difference between `p` and `*p`?
8. Why can a pointer parameter modify the caller's variable?
9. When are `arr[i]` and `*(arr + i)` equivalent?
10. Why is `arr[n]` invalid for an array of length `n`?
11. Why do we use `sizeof *p` in an allocation expression?
12. What is the difference between `malloc` and `calloc`?
13. What must be checked immediately after allocation?
14. Who should call `free`?
15. What is a memory leak?
16. Why is a pointer dangling after its allocation is freed?
17. Why is double free unsafe?
18. Why does assigning `NULL` after `free` help but not fix every alias?

Teacher Checklist

- The student separates value, address, pointer, and dereference correctly.
- The student traces pointer parameters without guessing.
- The student respects array bounds.
- The student writes a correct `malloc` or `calloc` size expression.
- The student checks for `NULL` before use.
- The student can state who owns the allocation.
- The student calls `free` exactly once and stops using the block afterward.
- The student can identify a leak, dangling pointer, invalid access, and double free.

Teaching Notes

- Keep the language in C for this course: `malloc`, `calloc`, `free`, and `NULL`.
- Do not introduce `realloc` yet; resizing belongs to Week 2.
- Ask students to mark the allocation's owner and lifetime directly on code traces.
- Do not let students run intentionally unsafe code merely to see what happens.

- Reward explanations of why code is safe, not only programs that appear to run.
- Use small arrays so students can draw every element and pointer relationship.

Key Teaching Philosophy

“Correct dynamic-memory code is not only code that allocates successfully. It is code that clearly controls size, ownership, valid access, and release.”